MASTER ARCHITECTURE

SHADOW OS v2

FOREX ENGINE PRO — Autonomous Trading Operating System

SINGLE SOURCE OF TRUTH

Version 1.0 · Sprint 1 · 2026-07-06

Classification:	Principal Architecture Document
Version:	1.0 — Active, enforced
Baseline:	SHADOW OS v1 / Architecture B
Scope:	5-year operational horizon, 100k+ trades, 10+ engines
Authors:	Sprint 0 (Foundation) + Sprint 1 (Runtime Awakening)
Next Review:	Sprint 2 (Domain Wiring)
Status:	Sprint 1 implemented · Sprints 2–5 specified

14	10	5	11	15
Components Mapped	Runtime Domains	Managers Defined	DB Tables Schemas	Sections Covered

SACRED CONSTRAINT — NEVER VIOLATED

No deployment, restart, or migration step may ever destroy the accumulated trading knowledge of the system.

1. Purpose and Scope

This document is the single source of truth for SHADOW OS v2. Every implementation decision, API design, and database schema must be consistent with what is written here. When this document and the code disagree, this document wins — update the code.

Design Horizon

5 years of continuous operation · 100,000+ closed trades · 10+ concurrent engines · Sub-second latency under any failure condition

2. Core Design Philosophy — Six Invariants

INVARIANT 1: Single Source of Truth

For every piece of information, exactly one layer owns it. Runtime Layer owns operational state. Memory Layer owns contextual memory. Knowledge Layer owns learned intelligence. Event Log owns historical record.

INVARIANT 2: Manager Mediation

No engine module reads from or writes to PostgreSQL directly. All DB access goes through the Manager Tier. Sprint 1 introduces RuntimeDomainManager; existing production files access DB directly until their manager is ready.

INVARIANT 3: Knowledge Immutability

Knowledge artifacts are never deleted; they are superseded. Every training run produces a new version. The lineage of how the system learned is always traceable.

INVARIANT 4: Memory Expiry

Memory entries have a defined lifecycle and expire naturally. The system does not rely on memory entries being present. Missing memory !' safe default behavior, not a crash.

INVARIANT 5: Recovery Completeness

After any failure, the Recovery Manager runs all 9 phases before trading resumes. System status is HALTED until RecoveryManager reports READY or DEGRADED.

INVARIANT 6: Financial Intent Atomicity

Every trade_open or trade_close is preceded by a committed trade_intent. No OANDA call is made without a committed PENDING intent. Violation !' ghost trades.

3. Four-Layer Memory Hierarchy

```
RUNTIME LAYER      Fast · Versioned · Domain-partitioned · Optimistic locking
  Write: ~5ms  Read: ~2ms  Durability: survives restart
  Domains: live, shadowA, shadowB, shadowC, shadowD, shadowM, exitLab, telemetry, scheduler, meta
  Owner: RuntimeDomainManager (Sprint 1)  !•  IMPLEMENTED

MEMORY LAYER       TTL-based · Contextual · Self-expiring · GC-managed
  Write: ~5ms  Read: ~3ms  Durability: TTL (hours to days)
  Namespaces: observations, cooldowns, market_state, volatility, correlations, decision_history
  Owner: MemoryManager (Sprint 3)  !•  PLANNED

KNOWLEDGE LAYER    Permanent · Versioned · Append-only · Checksum-verified
  Write: ~20ms  Read: ~10ms  Durability: forever (never deleted, only superseded)
  Artifacts: engineC/*, engineD/*, exitLab/*, market/*, system/*
  Owner: KnowledgeManager (Sprint 4)  !•  PLANNED

EVENT LOG          Immutable · Append-only · Audit-only
  Write: ~5ms  Durability: forever
  Purpose: compliance, analytics, replay — NOT recovery
  Owner: existing telemetry/index.js (unchanged)  !•  FROZEN
```

4. Component Map & Responsibilities

Component	File	Domain	Status
Live Bot	index.js	—	FROZEN
Server / Orchestrator	server.js	live, sched., meta	FROZEN
Shadow A/B/C/D	shadowlab.js	shadowA–D, exitLab	FROZEN
Shadow M	shadowm.js	shadowM	FROZEN
Telemetry	index.js	telemetry	FROZEN
RuntimeDomainManager	managers/RDM.js	ALL (10)	' Sprint 1
MemoryManager	managers/MM.js	memory_entries	Sprint 3
KnowledgeManager	managers/KM.js	knowledge_artifacts	Sprint 4
RecoveryManager	managers/Rec.js	consistency_log	Sprint 5
ValidationManager	managers/Val.js	consistency_log	Sprint 5

5. Runtime Domains (10 Domains)

Domain	Owner (current)	Owner (v2)	Write Freq.
live	server.js (direct)	LiveDomainAdapter (S2)	High — each trade
shadowA	shadowlab.js	ShadowLabAdapter (S2)	Each cycle ~30s
shadowB	shadowlab.js	ShadowLabAdapter (S2)	Each cycle ~30s
shadowC	shadowlab.js	ShadowLabAdapter (S2)	After training
shadowD	shadowlab.js	ShadowLabAdapter (S2)	Per 100 trades
shadowM	shadowm.js	ShadowMAdapter (S2)	Each poll
exitLab	shadowlab.js	ShadowLabAdapter (S2)	Each cycle
telemetry	telemetry/index.js	TelemetryAdapter (S2)	Each batch ~30s
scheduler	server.js	SchedulerAdapter (S2)	Each cycle boundary
meta	server.js	MetaAdapter (S2)	Boot/shutdown/status

6. Manager Hierarchy

Manager	Sprint	Key Methods	Tables
RuntimeDomainManager	1 '	createDomain, getDomain, compareAndSwap, takeSnapshot, rollback	runtime_domains, runtime_domain_history, system_snapshots, consistency_log
MemoryManager	3 #ó	set, get, delete, gc, listNamespace	memory_entries
KnowledgeManager	4 #ó	saveArtifact, loadArtifact, rollback, getHistory	knowledge_artifacts
RecoveryManager	5 #ó	runRecovery, assessDamage, repairDomain	consistency_log, ALL
ValidationManager	5 #ó	runCheck, autoRepair, schedule	consistency_log

7. Startup Sequence (Current)

```
node telemetry/server.js
  1. require('./index')      !’ db-adapter initialized
  2. require('./shadowlab')  !’ engines loaded
  3. require('./shadowm')    !’ Shadow M loaded
  4. restoreLiveState()      !’ reads events for open positions [FUTURE: RDM.getDomain('live')]
  5. Express API starts on PORT
  6. spawn('node', ['index.js']) !’ bot starts
  7. shadowM.start()         !’ polling loop [FUTURE: RDM manages shadowM domain]
  8. shadowLab cycle every 30s [FUTURE: RDM manages shadowA-D, exitLab domains]
  !’ SYSTEM OPERATIONAL

SHADOW OS v2 Full Startup (Sprint 5+):
  1. RecoveryManager.runRecovery() — 9 phases
  2. RuntimeDomainManager.init()   — validate tables
  3. RDM.getDomain('meta').bootCount++
  4. RDM.updateDomain('meta', { status: 'HEALTHY' })
  5. RDM.takeSnapshot('boot')
  !’ SYSTEM OPERATIONAL
```

8. Recovery Sequences

9-Phase Recovery (Sprint 5+)

Phase	Check	On Failure
1	DB Connectivity — pool.query('SELECT 1') with 10s timeout	HALT, retry 5×
2	Schema Integrity — all 11 tables present	HALT if runtime tables missing
3	Domain Integrity — all 10 domains, valid JSON	Repair with DEFAULT_DOMAINS
4	Event Cursor — shadowM.lastId vs events MAX(id)	Log WARN, use events as truth
5	Position Reconciliation — live.openTrades vs events	Use events-derived value
6	Knowledge Integrity — checksum verification	Rollback artifact, log CRITICAL
7	Memory GC — delete expired memory_entries	Non-fatal
8	Baseline Snapshot — takeSnapshot('post_recovery')	Log WARN if fails
9	Status Update — meta.status = HEALTHY or DEGRADED	Required final step

9. Database Schema Summary

Table	Sprint	PK	Purpose
events	Pre-v2	BIGSERIAL	Trade event log (immutable)
shadowm_trades	Pre-v2	BIGSERIAL	Shadow M trade tracking
shadowm_timeline	Pre-v2	BIGSERIAL	Shadow M pips timeline
runtime_domains	0 '	TEXT domain	Versioned domain state (10 rows)
trade_intents	0 '	BIGSERIAL	OANDA order idempotency
memory_entries	0 '	BIGSERIAL	TTL-based contextual memory
knowledge_artifacts	0 '	BIGSERIAL	Versioned learned intelligence
event_idempotency	0 '	TEXT key	Event deduplication
consistency_log	0 '	BIGSERIAL	Self-healing audit trail
system_snapshots	0 '	BIGSERIAL	Full state captures
runtime_domain_history	1 '	BIGSERIAL	Per-domain mutation history

10. Implementation Status

Component	Sprint	Status	Tests
DB Schema (10 tables)	0	' COMPLETE	19 pass
Dead code archive	0	' COMPLETE	—
Test framework	0	' COMPLETE	Operational
runtime_domain_history table	1	' COMPLETE	—
RuntimeDomainManager	1	' COMPLETE	107 pass
TradeIntentManager	2	#ó PLANNED	—
Domain Adapters	2	#ó PLANNED	—
MemoryManager	3	#ó PLANNED	—
KnowledgeManager	4	#ó PLANNED	—
RecoveryManager	5	#ó PLANNED	—
ValidationManager	5	#ó PLANNED	—

11. Sprint Roadmap

Sprint	Name	Core Deliverable	Key Risk
0 '	Foundation	DB schema, test framework, dead code archive	Idempotent migration
1 '	Runtime Awakening	RuntimeDomainManager — complete domain ownership	CAS pool deadlock (fixed)
2 #ó	Domain Wiring	Adapters — connect existing engines to RDM	Behavior regression
3 #ó	Memory OS	MemoryManager — TTL-based contextual memory	GC correctness
4 #ó	Knowledge OS	KnowledgeManager — learned intelligence persistence	Checksum integrity
5 #ó	Recovery OS	RecoveryManager + ValidationManager	Complex state repair
6 #ó	Intelligence	Incremental training, startup < 50ms at scale	Artifact size growth